

```
int sum = arr[0];
for (int i=1; i< arr.length; i++)
{
    arr[i] = arr[i] - sum;
    sum += arr[i];
}
```

	0	1	2	3	4	5	6
arr	3	4	9	6	8	4	2

עקבו בעזרת טבלת מעקב אחר ביצוע קטע התוכנית, ורשמו את תוכן המערך arr לאחר ביצוע קטע התוכנית. בטבלת המעקב יש לכלול עמודה עבור כל אחד מן המשתנים האלה: i, arr[i], sum.

i	i < 7	sum	arr[i]	חישוב
		3		
1	T	4	3 1	4 - 3 = 1
2	T	9	4 5	9 - 4 = 5
3	T	6	9 -3	6 - 9 = -3
4	T	8	6 2	8 - 6 = 2
5	T	4	8 -4	4 - 8 = -4
6	T	2	4 -2	2 - 4 = -2
7	F			

	0	1	2	3	4	5	6	length
arr	3	4	9	6	8	4	2	7
		1	5	-3	2	-4	-2	

```
int sum = arr[0];
for (int i=1; i< arr.length; i++)
{
    arr[i] = arr[i] - sum;
    sum += arr[i];
}
```

	0	1	2	3	4	5	6
arr	3	4	9	6	8	4	2

כתבו דוגמה למערך arr בגודל 5 שלאחר ביצוע קטע התוכנית כל התאים שלו יכילו מספרים חיוביים (כלומר, מספרים הגדולים מאפס).

before: [1, 2, 3, 4, 5]

after: [1, 1, 1, 1, 1]

במשחק קליעה למטרה, אדם יורה כמה חיצים למטרה. הניקוד עבור כל ירייה הוא מספר שלם מ־0 נקודות (החסאה מלאה) ועד 10 נקודות (פגיעה במרכז המטרה).
אדם מצליח במשחק אם מתיקיים לפחות אחד מן המקרים האלה:

- בממוצע הניקוד הוא מעל ל־5.0 נקודות.
- ברוב היריות הוא זכה ב־6 נקודות ומעלה.
- בירייה אחת או יותר הוא זכה ב־10 נקודות.

דוגמה: אדם ירה חמישה חיצים וזכה בניקוד: 3, 3, 9, 7, 5. בממוצע הוא זכה ב־5.4 נקודות $((3 + 3 + 9 + 7 + 5) / 5)$ ולכן הוא הצליח במשחק.

דוגמה נוספת: אדם ירה שישה חיצים וזכה בניקוד: 0, 8, 7, 1, 6, 7. בארבע יריות מתוך השש (כלומר, ברוב היריות שלו) הוא זכה ב־6 נקודות ומעלה, ולכן הוא הצליח במשחק.

דוגמה נוספת: אדם ירה שלושה חיצים וזכה בניקוד: 0, 10, 2. באחת מן היריות הוא זכה ב־10 נקודות ולכן הוא הצליח במשחק.

```
double d = 11 / 5; // 2.0
double d2 = ((double) 11) / 5; // 2.2
```

```
1 // 2.0
2 double d = 11 / 5;
3
4 double d2 = ((double) 11) / 5;
5
6 // 2.2
```

כתבו פעולה חיצונית ששמה success בשפת Java או Success בשפת C#, המקבלת מספר שלם – num. num מציין את מספר החיצים שהאדם ירה למטרה.
הפעולה תקלוט את מספר הנקודות שזכה בהן האדם בעבור כל חץ שהוא ירה.
הפעולה תחזיר true אם האדם הצליח במשחק, אחרת תחזיר הפעולה false.

```
1 public static boolean success(int num) {
2     double sum = 0.0;
3     int count6 = 0;
4     boolean tenPoints = false;
5     for (int i = 1; i <= num; i++) {
6         System.out.print("type score #" + i + ": ");
7         int score = scan.nextInt();
8         sum += score;
9         if (score >= 6) {
10             count6++;
11         }
12         if (score == 10) {
13             tenPoints = true;
14         }
15     }
16     double avgScore = sum / num;
17     if (avgScore > 5) {
18         return true;
19     }
20     if (count6 > num / 2) {
21         return true;
22     }
23     return tenPoints;
24 }
```


המחלקה **MyTime** מייצגת זמן, ולה שתי תכונות:

- myMinute – מספר שלם חיובי בין 0 ל־59 (כולל), המייצג דקה
- myHour – מספר שלם חיובי בין 7 ל־23 (כולל), המייצג שעה

הניחו שקיימות פעולות `get/Set` ו־`get/Set` עבור תכונות המחלקה, ופעולה בונה המקבלת ערכים עבור תכונות המחלקה.

לפניכם ממשק המחלקה. אפשר להשתמש בפעולות המחלקה בלי לממש אותן.

תיאור הפעולה	כותרת הפעולה
הפעולה מחזירה <code>true</code> אם זמן העצם הנוכחי מוקדם מזמן העצם <code>other</code> , אחרת (כלומר אם הזמן שווה לו או מאוחר ממנו) הפעולה מחזירה <code>false</code> .	<code>Java – public boolean before (MyTime other)</code> <code>C# – public bool Before (MyTime other)</code>
הפעולה מחזירה את ההפרש בדקות בערך מוחלט בין זמן העצם הנוכחי ובין זמן העצם <code>other</code> .	<code>Java – public int diff (MyTime other)</code> <code>C# – public int Diff (MyTime other)</code>

המחלקה **Parking** מציגה מידע על מכונית שחנתה בחניון, ולה שלוש תכונות:

- id – מספר לוחית רישוי של המכונית, מטיפוס מחרוזת
- in – זמן כניסת המכונית לחניון, מטיפוס `MyTime`
- out – זמן יציאת המכונית מן החניון, מטיפוס `MyTime`

הניחו שקיימות פעולות `get/Set` ו־`set/Set` עבור תכונות המחלקה, ושתכונות המחלקה אינן `null`.

נתון מערך חד־ממדי `cars` מטיפוס **Parking** המכיל מידע על כל המכוניות שחנו בחניון ביממה מסוימת (החניון נפתח

ב־07:00 ונסגר ב־23:59). המערך אינו ממוין ואינו מכיל ערכי `null`.



ב. כתבו פעולה פנימית ששמה `improving` בשפת `Java` או `Improving` בשפת `C#`, המחזירה `true` אם סך ההפרשים בימים שנחשבים "שימור" גדול מסך ההפרשים בימים שנחשבים "הרעה". אחרת הפעולה תחזיר `false`.

למשל, בדוגמה שלעיל הפעולה תחזיר `true` כיוון שסך ההפרשים בימים שנחשבים "שימור" גדול מסך ההפרשים בימים שנחשבים "הרעה" ($17 > 14$).

חובה להיעזר בפעולה שכתבתם בסעיף א.

```
1 class Results {
2     private int[] arr;
3     private String name;
4
5     public int deltas(boolean isLonger) {
6         ...
7         return sum;
8     }
9
10    public boolean improving() {
11        return deltas(false) > deltas(true);
12    }
13 }
```



א. כתבו פעולה חיצונית ששמה `first` בשפת `Java` או `First` בשפת `C#`, המקבלת את המערך `cars`, ומדפיסה את מספר לוחית הרישוי של המכונית שנכנסה ראשונה לחניון.

הניחו שאין שתי מכוניות שנכנסו לחניון בדיוק באותו הזמן.

```
1
2 Parking[] cars = new Parking[] {
3     new Parking("11-111-11", new MyTime(12,30), new MyTime(14,15)),
4     new Parking("22-222-22", new MyTime(11,30), new MyTime(13,45)),
5     new Parking("33-333-33", new MyTime(7,10), new MyTime(10,10)),
6     new Parking("44-444-44", new MyTime(9,15), new MyTime(11,30)),
7 };
8
9 first(cars);           // 33-333-33
10
11 class MyTime {
12     private int myHour;
13     private int myMinute;
14
15     פונקצית עזר: מחזיר אמת עם אני לפני האחר //
16     public boolean before(MyTime other) {
17         if (myHour == other.myHour) {
18             return myMinute < other.myMinute;
19         }
20         return myHour < other.myHour;
21     }
22 }
```



א. כתבו פעולה חיצונית ששמה `first` בשפת `Java` או `First` בשפת `C#`, המקבלת את המערך `cars`, ומדפיסה את מספר לוחית הרישוי של המכונית שנכנסה ראשונה לחניון.

הניחו שאין שתי מכוניות שנכנסו לחניון בדיוק באותו הזמן.

```
1 class MyTime {
2     private int myHour;
3     private int myMinute;
4
5     פונקצית עזר: מחזיר אמת עם אני לפני האחר //
6     public boolean before(MyTime other) {
7         ...
8     }
9 }
10
11 public static void first(Parking[] cars) {
12     int min = 0;
13     for (int i = 1; i < cars.length; i++) {
14         if (cars[i].getIn().before(cars[min].getIn())) {
15             min = i;
16         }
17     }
18     System.out.println(cars[min].getId());
19 }
```




המחיר בחניון נקבע על פי משך החניה. בשעתיים הראשונות (120 דקות) החניה חינם והחל מהדקה ה-121 התעריף הוא שקל אחד לכל דקת חניה.

ב. (1) כתבו פעולה פנימית במחלקה Parking ששמה total בשפת Java או בשפת C# המחזירה את סך כל מספר הדקות שהמכונית חנתה בחניון.

```
1 MyTime t1 = new MyTime(13,30);
2 MyTime t2 = new MyTime(14,45);
3
4 print(t1.diff(t2));
5 print(t2.diff(t1));
6
7 class MyTime {
8     private int myHour;
9     private int myMinute;
10
11     // פונקציה עזר: מחזיר זמן (דקות) שחלף ממני עד האחר
12     public int diff(MyTime other) {
13         return Math.abs((60*myHour + myMinute) -
14                         (60*other.myHour + other.myMinute));
15     }
16 }
17
18 class Parking {
19     private String id;
20     private MyTime in;
21     private MyTime out;
22
23     public int total() {
24         return this.out.diff(this.in);
25     }
26 }
```



המחיר בחניון נקבע על פי משך החניה. בשעתיים הראשונות (120 דקות) החניה חינם והחל מהדקה ה-121 התעריף הוא שקל אחד לכל דקת חניה.

ב. (2) כתבו פעולה חיצונית ששמה sumMoney בשפת Java או SumMoney בשפת C#, המקבלת את המערך cars, ומחזירה את הסכום הכולל ששילמו כל בעלי המכוניות. אפשר להשתמש בפעולה שכתבתם בסעיף ב(1).

```
1 Parking[] cars = new Parking[] {
2     new Parking("11-111-11", new MyTime(12,30), new MyTime(14,15)),
3     new Parking("22-222-22", new MyTime(11,30), new MyTime(13,45)),
4     new Parking("33-333-33", new MyTime(7,10), new MyTime(10,10)),
5     new Parking("44-444-44", new MyTime(9,15), new MyTime(11,30)),
6 };
7
8 print(sumMoney(cars)); // 90
9
10 public static int sumMoney(Parking[] cars) {
11     int sum = 0;
12     for (Parking car: cars) {
13         int mins = car.total();
14         if (mins > 120) {
15             sum += mins - 120;
16         }
17     }
18     return sum;
19 }
```

כדי להציג מסר כלשהו, מערבלים את סדר התווים שמרכיבים אותו (התווים עצמם נשמרים אך מיקומם משתנה). לדוגמה: לאחר ערבול סדר התווים של המילה World אפשר לקבל את המסר המוצגן rdoWl.

כדי שמקבל המסר המוצגן יוכל לפענח את המסר המקורי של השולח, מצורף למסר המוצגן "מערך מפענח". גודל המערך המפענח הוא בגודל המסר המוצגן, ובכל תא מופיע מספר האינדקס המקורי של כל תו. לדוגמה: עבור המסר המוצגן rdoWl (של המסר המקורי World), המערך המפענח הוא:

0	1	2	3	4
2	4	1	0	3

הסבר: התו הראשון במסר המוצגן (r) מופיע במסר המקורי באינדקס 2, התו השני במסר המוצגן (d) מופיע במסר המקורי באינדקס 4, התו השלישי במסר המוצגן (o) מופיע במסר המקורי באינדקס 1, וכן הלאה.

0	1	2	3	4
W	o	r	l	d

0	1	2	3	4
r	d	o	W	l

0	1	2	3	4
2	4	1	0	3



א. רשמו את המערך המפענח עבור המסר Study, שהוצגן למסר dyutS.

0	1	2	3	4
S	t	u	d	y

0	1	2	3	4
d	y	u	t	S

0	1	2	3	4

3. כתבו פעולה חיצונית ששמה originalText בשפת Java או OriginalText בשפת C#, המקבלת מחרוזת מוצפנת - str ומערך מפענח - arr. הפעולה תחזיר את המחרוזת המקורית.

```

1 print(arrToString(new char[] { 'w', 'o', 'r', 'l', 'd' })); //
2
3 // פונקציה עזר: מקבלת מערך של תווים
4 // ומחזירה אותם כמחרוזת
5 public static String arrToString(char[] chars) {
6     String str = "";
7     for (int i = 0; i < chars.length; i++) {
8         str = str + chars[i];
9     }
10    return str;
11 }
12
13 public static String originalText(String str, int[] arr) {
14     char[] chars = new char[arr.length];
15     for (int i = 0; i < arr.length; i++) {
16         int indx = arr[i];
17         chars[indx] = str.charAt(i);
18     }
19     return arrToString(chars);
20 }

```